Software Architecture

Contents

1. Architecture and design

The most useful definition comes from Martin Fowler, and it is about cost rather than scope: **architecture is the set of decisions that are expensive to change later.** Design is everything else.

That test is more helpful than trying to draw a line by size or altitude, because it puts the question where it belongs. How much would it cost to change this in a year?

Usually architectural	Usually design
Where the boundaries between deployable units sit	How a class inside a module is structured
Which data store each part owns	Which JSON library is used
Whether components talk synchronously or through events	The name of a method
Which consistency guarantee an operation gets	Whether a loop or a stream is clearer
Where failure is isolated	How a single query is written

The categories are not fixed. A library choice becomes architectural when it leaks into every signature in the codebase, and a boundary becomes ordinary design when the tooling makes it cheap to move.

One terminology note, since the words get used loosely and occasionally tested. An **architectural style** is a broad set of structural constraints, such as layered, hexagonal, event-driven or microservices. An **architectural pattern** is a reusable solution to a recurring architectural problem, such as CQRS, the strangler fig or the transactional outbox. The industry is not consistent about the distinction, so it is worth knowing and not worth arguing about.

Two framings worth carrying into an interview.

Architecture is not a phase. It is not decided once at the start and then implemented. Section 17 covers what replaces that model.

The goal is not the best design, it is the cheapest correction. Since some decisions will be wrong, the valuable property is that the wrong ones can be changed. That is why "which of these decisions is hard to reverse, and can I defer it" is a stronger question than "which option is best".

2. Drivers and quality attributes

An architecture is the result of satisfying a set of constraints and accepting the trades between them. Naming those **drivers** before choosing anything is what stops a design being a preference:

- Functional requirements, meaning what the system has to do.

- Quality attributes, meaning how well, which is the rest of this section.
- Organisational structure, which section 9 shows will assert itself regardless.
- Regulatory and contractual constraints, such as data residency or retention.
- Existing technology and integrations that cannot be replaced.
- Team expertise, since an architecture nobody can operate is not a good one.
- Deployment and operating model, including who runs it and how often it ships.
- Cost, both to build and to keep running.

Quality attributes are also called non-functional requirements, or the *-ilities*. Functional requirements say what the system does; quality attributes say how well, and they are what architecture actually trades between.

Attribute	The question it answers
Performance	How fast is one operation
Scalability	What happens when load multiplies
Availability	How much downtime is acceptable
Reliability	How consistently does it perform correctly over time
Maintainability	How long does a change take
Testability	Can a part be verified without the whole
Security	Who can reach what
Observability	Can a problem be diagnosed from outside
Deployability	How safely and often can it ship
Cost	What does running it consume

They conflict, and that is the point. Architecture is mostly the practice of choosing which to favour:

- Availability against consistency **during a network partition**, which is the CAP trade from the distributed systems note, and is narrower than a general trade between the two.
- Performance against maintainability, since an abstraction that makes change cheap usually adds indirection.
- Scalability against simplicity, since a system that scales to a hundred nodes is harder to run at three.
- Security against convenience, in almost every case.
- Cost against all of them.

The discipline that makes these usable is **stating them as measurements**. "Highly available" is not a requirement; "99.9% of requests succeed over a rolling 30 days" is. The system design note covers turning these into service level objectives, and the point here is that an unmeasurable quality attribute cannot be traded against anything, so it ends up decided by accident.

3. Architecture decision records

An **ADR, Architecture Decision Record**, is a short document capturing one decision: what was decided, why, and what it costs. They are numbered and kept in the repository beside the code, and an accepted one is treated as a historical record: when the decision changes, a new record supersedes it rather than the old one being rewritten.

The reason they earn their place: **the decision survives, and the reasoning evaporates.** Six months later the constraint that made an option obvious is forgotten, so nobody can tell whether it still applies. Without that, a team either preserves a decision nobody understands, or reverses it and rediscovers the original problem.

```
# 0012. Use Postgres for the order service

Status: Accepted
Date: 2026-08-16
Supersedes: none

## Context
Orders need multi-row transactions and reporting queries that are not
known in advance. Volume is well under a million rows per day. The team
already runs Postgres and has no Cassandra experience.

## Decision
Use Postgres as the order service's primary store.

## Consequences
Transactions and ad-hoc reporting come for free. A single writer caps
write throughput, and if volume grows beyond roughly ten thousand writes
per second this decision needs revisiting. Operational load is unchanged
because Postgres is already run for other services.
```

The section that separates a real ADR from a sales document is **Consequences**, and it has to include the bad ones. A record listing only benefits tells a future reader nothing about when to reconsider.

4. Layered architecture

The most common structure, and usually the right one for an ordinary application.

```
Presentation      controllers, request and response models
      ↓
Application       use cases, transaction boundaries, orchestration
      ↓
Domain            entities, business rules
      ↓
Persistence       repositories, database access
```

The rule is that dependencies point downward and each layer talks only to the one beneath it. Strict layering forbids skipping a layer; relaxed layering allows presentation to reach persistence directly for a simple read, which is pragmatic and erodes quickly if nobody is watching.

Its strengths are real: everyone recognises it, code is easy to place, and the structure survives people joining and leaving.

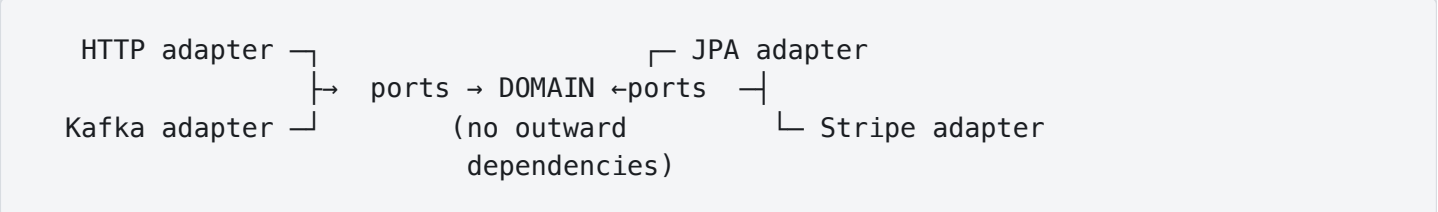
Its weakness shows up in the common database-centric form, where dependencies flow downward the whole way and **the domain ends up depending on persistence**, either directly or through a repository class it does not own. Business rules can then no longer be compiled, tested or reasoned about without the database, so every test of a rule becomes an integration test and swapping the store means touching the domain.

That is a property of that particular arrangement rather than of layering itself. A layered design can invert the bottom dependency and have the domain own the repository interface, which is exactly what the next section formalises.

5. Hexagonal architecture

Also called **ports and adapters**, named by Alistair Cockburn. **Onion architecture** and **Clean architecture** are close relatives, and the differences between the three are mostly vocabulary rather than substance.

One rule: **dependencies point inward.** The application core, holding the domain model and the use cases, depends on abstractions it owns rather than on infrastructure. It still depends on the language's standard library and on its own domain types; what it does not depend on is anything outside the boundary. Everything out there, whether a database, an HTTP handler, a message broker or a third-party API, is an adapter plugged into a **port** that the core defines.



In this example the port is an interface owned by the application core, and the adapter implements it:

```java
// Port. Lives with the domain, named for what the domain needs.
public interface OrderRepository {
    Optional<Order> findById(OrderId id);
    void save(Order order);
}

// Adapter. Lives in infrastructure, depends inward on the domain.
@Repository
class JpaOrderRepository implements OrderRepository { ... }
```

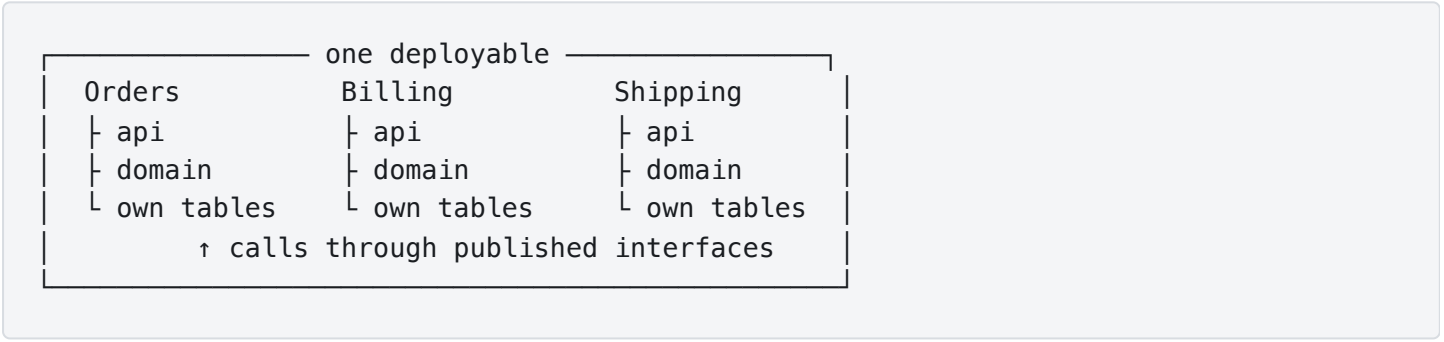
That is the dependency inversion principle from the design principles note, applied at the scale of the whole application rather than one class.

What it buys: the core is testable with no infrastructure at all, adapters can be replaced without touching business rules, and the arrangement forces the domain to be defined in its own terms rather than the database's.

What it costs: more indirection, and usually a mapping layer between domain objects and persistence models, which is real work that produces no features. **For a service that is mostly create, read, update and delete over a few tables, this is over-engineering**, and saying so is a better answer than applying it everywhere.

6. The modular monolith

One deployable unit, divided into modules with boundaries as strict as service boundaries: each module owns its data, exposes an explicit interface, and never reaches into another module's tables.



The discipline that makes it work is the one rule: **no module reads another module's tables.** Communication goes through a published interface or an in-process event. Enforcing that with a build-time check rather than good intentions is what section 17 is about.

It is worth arguing for in an interview, because it gets most of what people want from microservices without the distributed systems tax. Boundaries are explicit, teams can own modules, and reasoning is local. What it does not give is independent deployment or independent scaling, which are among the main reasons teams adopt microservices.

It is also the **right starting point**, because module boundaries can be moved with a refactor while service boundaries need a migration. The boundaries that turn out to be correct are usually not the ones drawn on day one, and the cheapest way to find them is to build the system and watch where changes cluster.

7. Monolith and microservices

	Monolith	Microservices
Deployment	One unit, one release	Independent per service
Scaling	The whole application together	Per service, where the load is

	Monolith	Microservices
Calls between parts	In-process, reliable	Over a network, with partial failure
Transactions across parts	Ordinary database transactions	A saga, since there is no distributed transaction
Data	Usually one schema	Each service owns its data, and nothing else reads it directly
Technology	Uniform	Chosen per service
Refactoring across boundaries	An ordinary change	A coordinated migration
Debugging	One stack trace	Distributed tracing
Operational load	One thing to run	Multiplied by service count

Two rows are worth pulling out, because they get conflated. A **saga** coordinates a business transaction spanning several services, with compensating steps when one of them fails. The **transactional outbox** solves a different problem entirely: writing a database change and the event announcing it atomically, inside *one* service. A saga typically uses an outbox to publish its steps reliably, so they work together, but the outbox is not a substitute for the saga and neither is a distributed transaction. The distributed systems note covers both.

The framing that holds up under questioning: **microservices are an organisational solution with a technical cost.** They exist so that teams can deploy without coordinating with each other. Most of the operational and consistency properties in the table get harder, while fault isolation, independent scaling and technology choice get better, and the trade is worth it when the coordination cost between teams exceeds the distributed systems cost.

SOA, Service-Oriented Architecture, came first and is worth a sentence because the comparison gets asked directly. SOA services tend to be larger, organised around enterprise capabilities, with centralised governance and often an enterprise service bus carrying orchestration and transformation. Microservices are smaller, own their data, deploy independently, and put the logic in the services rather than the pipes. A common microservices principle is "smart endpoints, dumb pipes", whereas traditional SOA often places more responsibility in shared integration infrastructure such as an enterprise service bus.

That has a direct consequence: with one team, most of the benefit does not apply and all of the cost does.

Prerequisites worth naming before splitting anything:

- Automated deployment, because the deployment count is about to multiply.
- Distributed tracing and centralised logs, because a stack trace no longer covers one request.
- A clear on-call model, because failures are now between services rather than inside one.
- A way to run a meaningful subset locally, or productivity falls sharply.

Without those, splitting produces a distributed monolith: the coupling of a monolith with the operational cost of microservices, which is the worst combination available.

8. Finding service boundaries

The genuinely hard part, and the reason so many microservice migrations disappoint. **Bad boundaries are worse than no boundaries**, because a change that would have been one commit becomes three services, three reviews and a coordinated release.

Signals of a boundary worth drawing:

- It maps to a **business capability**, something the organisation would recognise as a thing it does.
- It **owns its data**, with no other service reading its tables.
- It **changes for its own reasons**, which is the single responsibility principle at system scale.
- Most changes touch **one** service.
- It can be **deployed without coordination**.

Boundaries that go wrong, and what they look like:

Anti-pattern	Symptom
Split by technical layer	A "validation service" or "database service" that every request passes through
Entity service	A "user service" everything calls for everything, so it is on every critical path
Boundaries needing distributed transactions	Ordinary operations span services and need a saga to be correct
Chatty boundaries	Two services that call each other constantly, which means they are one service

The test worth remembering: **if implementing a typical feature routinely means changing three services and coordinating their releases, that is a strong signal the boundaries are wrong.** Any single feature can legitimately cross services; it is the frequency that carries the information. That is measurable rather than aesthetic, and it is what to watch after a split rather than before it.

9. Conway's Law and team shape

Melvin Conway, 1967: organisations design systems that copy their own communication structure.

It is an observation rather than advice, and it holds. Three teams produce three components. A team split across time zones produces a component boundary at the time zone. A shared database persists because two teams both need it and neither owns it.

The consequence is that **architecture and organisation cannot be designed separately.** An architecture that cuts across team boundaries will be eroded by the daily cost of coordinating, until it matches the organisation again.

The **inverse Conway manoeuvre** is the deliberate use of this: change the team structure to get the architecture wanted, rather than fighting the existing structure. Reorganising into teams that own business capabilities tends to produce services around business capabilities.

Team Topologies is the vocabulary worth knowing here, and it names four kinds of team: **stream-aligned** teams that own a flow of work end to end, **platform** teams that provide the paved path, **enabling** teams that raise capability elsewhere, and **complicated-subsystem** teams for parts needing deep specialism. Its central constraint is **cognitive load**: a team can only own as much as it can hold in its head, and a boundary that exceeds that will decay regardless of how correct the diagram is.

10. Ubiquitous language and bounded contexts

DDD, Domain-Driven Design, from Eric Evans in 2003, is a set of tools for building software around the business domain rather than around technology. Two of its ideas are architectural and worth knowing well.

Ubiquitous language is one vocabulary, shared by engineers and domain experts, used in conversation, documentation and code. When the business says "policy" and the code says "contract", every discussion pays a translation cost and the bugs live in the gap between the two words. The test is whether a domain expert could read a method name and recognise it.

A bounded context is the boundary within which one model and its language are consistent. This is the idea that does the most architectural work, because it explains why a single shared model across a large organisation so often becomes a source of coupling.

"Customer" is the standard example. To **sales** a customer is a prospect with a pipeline stage. To **billing** a customer is a payment method and an address. To **support** a customer is a history of tickets. Building one Customer class for all three produces something with forty fields, of which each consumer uses eight, and which cannot be changed without agreement from everyone.

Three models, one per context, each small and each correct in its own context, is the answer. They refer to the same real person by identifier, and they share nothing else.

Bounded contexts are the best available candidates for service boundaries, which is the practical link between DDD and section 8. A boundary drawn where the language changes tends to satisfy the signals in that section without being forced to.

11. Aggregates

An **aggregate** is a cluster of objects treated as one unit for changes, with one entity as its **root**. Three rules define it:

- External code holds a reference only to the root, never to something inside.
- Prefer a transaction that changes one aggregate.
- Any rule that must hold at all times lives inside a single aggregate.

The second is a design guideline rather than something the database enforces. A transaction can technically span several aggregates, and the reason to avoid it is that the aggregate is meant to *be* the consistency boundary, so a transaction crossing two of them usually means the boundary is drawn in the wrong place.

The third rule is the design tool, and it is the reason aggregates matter to architecture rather than only to code. **If an invariant must be enforced atomically across two pieces of state, they are strong candidates for the same aggregate.** If eventual consistency between them is acceptable, they can be separate aggregates. Whether they should then also be separate *services* is a different question, answered by section 8, since separate aggregates live perfectly well inside one service.

```
public class Order {                    // aggregate root
    private final OrderId id;
    private final List<OrderLine> lines = new ArrayList<>();
    private OrderStatus status;

    // The invariant lives here, where every change passes through it.
    public void addLine(Product product, int quantity) {
        if (status != OrderStatus.DRAFT) {
            throw new IllegalStateException("cannot change a submitted order");
        }
        if (total().plus(product.price().times(quantity)).isGreaterThan(creditLimit))
{
            throw new CreditLimitExceededException(id);
        }
        lines.add(new OrderLine(product.id(), quantity, product.price()));
    }

    public List<OrderLine> lines() { return List.copyOf(lines); }   // no direct
access
}
```

Two supporting distinctions:

An entity has an identity that persists through change. An `Order` is the same order after its lines change. **A value object** is defined entirely by its attributes and has no identity: `Money`, an `Address`, a `DateRange`. Value objects should be immutable, and making them explicit types rather than raw `BigDecimal` and `String` removes a whole category of bug.

Aggregates should be **small**. The instinct to draw a large one, with an order holding its customer holding their whole history, produces a transaction boundary that locks half the database.

12. Context mapping

Once there is more than one bounded context, the relationships between them need naming. DDD gives them names, and the vocabulary is genuinely useful in a design discussion.

Relationship	What it means
Shared kernel	Two contexts share part of a model. Tight coupling, and every change needs agreement
Customer and supplier	Upstream serves downstream, and downstream has influence over what it gets
Conformist	Downstream adopts upstream's model as it is, with no influence
Anticorruption layer	Downstream translates upstream's model into its own, protecting itself from it
Open host service	Upstream publishes a stable interface designed for many consumers
Published language	A shared interchange format the contexts agree on
Separate ways	No integration at all, because the cost exceeds the value

The anticorruption layer is the one to know best. When integrating with a legacy system, a third-party API, or another team's model that we cannot influence, the choice is to let their concepts into our domain or to translate at the boundary. Translating costs a mapping layer and keeps the domain clean; not translating means their model spreads through our code and their next change reaches everywhere.

It is the Adapter pattern from the design patterns note, at architectural scale, and the payment gateway example there is exactly this shape.

13. Event-driven architecture

Components communicate by publishing events describing what happened, rather than calling each other directly.

The benefits are decoupling in several dimensions at once. The producer does not know who consumes, so a new consumer is added without touching it. The consumer does not have to be available when the event is published. Bursts buffer rather than overwhelm.

The costs are equally structural. **The flow stops being readable in one place**: understanding what happens after an order is placed means searching for consumers rather than following a call stack. Debugging needs distributed tracing. Where events are processed asynchronously, which is the usual arrangement, consistency between producer and consumer becomes eventual and has to be treated as a product decision rather than only a technical one. And event schemas are contracts, so changing one has the same compatibility problem as changing an API, without the version in the URL to make it obvious.

Three things get called event-driven and are meaningfully different, which is worth separating because interviewers conflate them:

Style	The event carries	Trade
Event notification	Only that something happened, and an identifier	Small messages, but consumers call back for detail, so coupling to the producer's API remains
Event-carried state transfer	The data the consumer needs	Consumers become autonomous, at the cost of larger messages and duplicated data
Event sourcing	The events *are* the stored state	Section 15, and a much larger commitment

Moving from notification to state transfer is the usual way to remove a chatty callback, and the price is that consumers now hold copies of data they do not own.

14. CQRS

CQRS, Command Query Responsibility Segregation: use a different model for writing than for reading.

The minimal version is two models in the same database and two code paths, one for commands and one for queries. The full version has a separate read store, updated asynchronously from the write side, shaped exactly for the queries it serves.

```
Command → write model → database
                            │  events or change feed
                            ↓
                        read model → optimised for queries
```

It pays when the read and write shapes genuinely diverge. A normalised write model that guarantees invariants is often a poor shape for a dashboard needing six joins, and denormalising the write model to suit the dashboard damages the thing the write model exists for.

The costs, in order of how often they cause regret:

- **Two models to maintain**, so every change touches both.
- **Eventual consistency becomes user-visible** when the read store updates asynchronously. The user saves something and does not see it in the list, which is a product problem rather than a technical one, and it needs a deliberate answer.
- **More moving machinery** to operate and monitor.

Two clarifications worth having ready, because they are commonly assumed:

CQRS does not require event sourcing, and event sourcing does not require CQRS. They appear together often, which is why they are confused.

CQRS does not require separate databases. Separating the models is the pattern; separating the stores is one implementation of it, and the expensive one.

CQRS is about separating the models, not about scaling reads and writes independently. Independent scaling becomes possible once the stores are separate, and it is a consequence rather than the definition.

15. Event sourcing

Store the sequence of events that happened, rather than the current state. Current state is derived by replaying them.

```
Instead of:   balance = 120

Store:        AccountOpened
              Deposited 100
              Deposited 50
              Withdrew 30      →  replay gives 120
```

What it buys is genuine and specific: **a complete audit log by construction**, since the events are the record rather than something written alongside it; **temporal queries**, because the state at any past moment can be rebuilt; and **new projections retroactively**, since a question nobody thought to ask can be answered from history.

What it costs is larger than most teams expect:

- **Querying needs projections.** There is no table to select from, so every read shape has to be built and maintained.
- **Event schemas are forever.** An event written three years ago must still be readable, so versioning and upcasting become permanent work.
- **Deleting data conflicts with an append-only log**, which collides directly with data protection rules requiring erasure. The usual answers, such as encrypting per subject and discarding the key, are extra machinery.
- **Performance needs snapshots** once an aggregate has thousands of events, since replaying from the beginning stops being viable.

The honest position, and a good interview answer: event sourcing fits domains where **the sequence of events is the truth** rather than a derivative of it, such as accounting or trading, and particularly where reconstructing historical state is a core requirement rather than a nice property. A regulatory audit obligation on its own does not imply it, since an append-only audit table, immutable logs or database change history usually satisfy that at a fraction of the cost. Elsewhere it is a large permanent commitment for benefits a change log would have delivered.

16. Communicating architecture with C4

Simon Brown's **C4 model** solves a real problem: one diagram cannot serve every audience, so drawing four, at different levels of zoom, works better than drawing one that tries to.

Level	Shows	Audience
1. Context	The system, its users, and the external systems it talks to	Anyone, including non-technical
2. Container	Separately runnable applications and data stores, and how they communicate	Engineers and operations
3. Component	The major parts inside one container	Engineers working in that container
4. Code	Classes and their relationships	Rarely drawn, since the code says it better

The word **container** here predates Docker and does not mean a Docker container. It means a separately runnable or deployable thing: an application, a database, a message broker, a single-page application.

Most value is in the first two. Level 1 answers "what is this and who uses it" in a picture anyone can read, and level 2 answers "what actually runs and what talks to what", which is the diagram most teams are missing.

Three rules make any architecture diagram useful, independently of the model:

- **Every box says what it is and what technology it uses.** "Orders" is not a box; "Orders service, Java and Spring Boot" is.
- **Every line says what flows and how.** An unlabelled arrow carries no information. "Reads order history, JSON over HTTPS" does.
- **There is a legend**, because notation nobody shares is decoration.

17. Evolutionary architecture

The alternative to deciding architecture once, up front, and then discovering it was wrong.

The stance: **make reversible decisions quickly and irreversible ones carefully, and spend effort converting the second kind into the first.** A decision behind an interface is more reversible than one spread through every file. A boundary inside one deployable is more reversible than one between two services. Deferring an irreversible decision until the information exists is usually worth more than making it early and well.

Fitness functions are the mechanism that makes this safe. A fitness function is an automated check that the architecture still holds, run in the build like any other test. In Java, ArchUnit expresses structural rules directly:

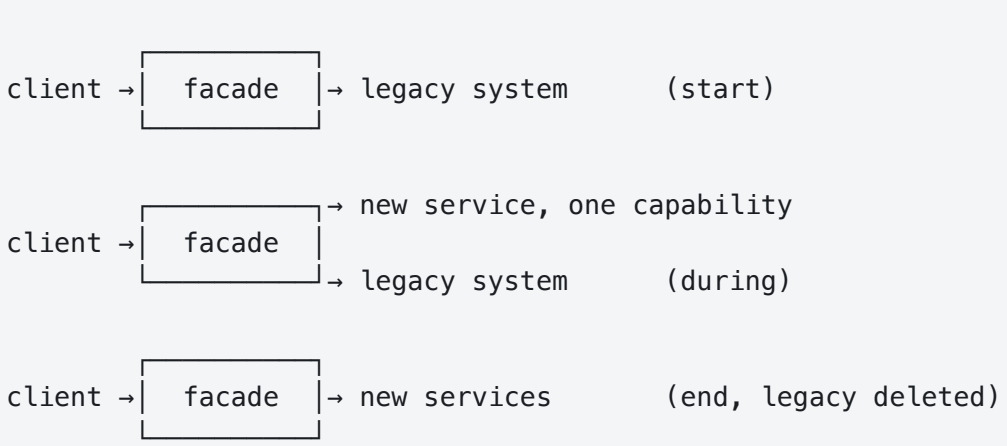
```
@AnalyzeClasses(packages = "com.example.orders")
class ArchitectureTest {

    // Hexagonal architecture's one rule, enforced rather than hoped for.
    @ArchTest
    static final ArchRule domain_depends_on_nothing_outward =
            noClasses().that().resideInAPackage("..domain..")
                    .should().dependOnClassesThat()
                    .resideInAnyPackage("..infrastructure..", "..web..");

    // The modular monolith's one rule.
    @ArchTest
    static final ArchRule modules_do_not_reach_into_each_other =
            noClasses().that().resideInAPackage("..orders..")

.should().dependOnClassesThat().resideInAPackage("..billing.internal..");

    @ArchTest
    static final ArchRule no_cycles =
            slices().matching("com.example.(*)..").should().beFreeOfCycles();
}
```

Not every quality attribute is structural, and fitness functions cover others too: a performance test that fails past a latency budget, a check that no dependency has a known vulnerability, a limit on deployable size.

Architecture erosion is the gap between the intended architecture and the real one. It grows through ordinary pressure, one reasonable shortcut at a time, and it is invisible until someone tries to make a change the architecture was supposed to make easy. A rule written in a document decays. The same rule in the build does not, which is the entire argument for fitness functions.

18. Strangler fig migration

Named by Martin Fowler after the plant that grows around a tree and eventually replaces it. It is the standard alternative to replacing a legacy system all at once.



The steps: put a facade in front of the existing system so callers stop depending on it directly; pick one capability with a clean seam; implement it in the new system; route a fraction of traffic to it and verify the results match; route all of it; delete the old path. Then repeat.

Why this beats replacing everything at once: a full rewrite produces no value until it is finished, competes against a system that keeps changing underneath it, and concentrates all the risk at the end, which is the worst possible place for it. A strangler migration delivers value continuously and can be stopped part-way without the work being wasted.

Data is the hard part, and it is where these migrations actually get difficult. During the transition both systems usually need the same data, so the options are shared tables, which couples them, synchronisation, which risks divergence, or making one side the owner and the other a consumer of its events, which is cleanest and slowest.

The trap worth naming is the **naive dual write**, where the facade writes to both systems and neither owns the data. There is no transaction across the two, so a failure on either side leaves them disagreeing with nothing to say which is right, and the divergence is usually discovered much later by a customer. Pick an owner, derive the other side through a controlled synchronisation mechanism, often events but a change feed or batch job can serve, and run a reconciliation that reports differences rather than assuming there will not be any. Choosing that ahead of time is worth more than any amount of planning for the code.

19. Interview questions

Each answer below is written to be said out loud as it stands. The last sentence always leads toward the next question rather than closing the topic.

"What is the difference between architecture and design?"

> The definition I use is Fowler's, which is about cost rather than scope: architecture is the set of decisions that are expensive to change later, and design is everything else. That test is more useful than trying to split them by size, because it puts the question where it belongs, which is how much this would cost to change in a year. So where the boundaries between deployable units sit is architectural, and how a class inside a module is structured is design, but a library choice can become architectural if it leaks into every signature. The framing I would add is that the goal is not the best design, it is the cheapest correction, because some decisions will be wrong and what matters is whether they can be changed.

"When would you use microservices?"

> When the cost of teams coordinating with each other is larger than the cost of distributed systems, which makes it an organisational decision with a technical price rather than a technical upgrade. Many operational and consistency properties get harder: in-process calls become network calls with partial failure, cross-service transactions become sagas, one stack trace becomes distributed tracing, and operational load multiplies by the number of services. Some things do get better, though, which is fault isolation, independent scaling and being able to choose technology per service. What they buy is independent deployment and independent scaling, so with one team most of the benefit does not apply and all of the cost does. I would start with a modular monolith, because module boundaries can be moved with a refactor while service boundaries need a migration, and the boundaries that turn out to be right are usually not the ones drawn on day one. I would also want deployment automation, tracing and a clear on-call model in place first, because without those a split produces a distributed monolith.

"How do you decide where to split services?"

> By business capability rather than technical layer, and the signals I look for are that the candidate owns its own data with nothing else reading its tables, that it changes for its own reasons, and that it can be deployed without coordinating with anything else. Bounded contexts from domain-driven design are the best available starting point, because a boundary drawn where the language changes tends to satisfy those signals naturally: what "customer" means to billing is genuinely not what it means to support, and that difference is a real seam rather than an invented one. The anti-patterns I watch for are splitting by layer, so a validation service everything passes through, and entity services like a user service that ends up on every critical path. The test afterwards is measurable: if a typical feature routinely means changing three services and coordinating their releases, that is a strong signal the boundaries are wrong. Any one feature can legitimately cross services, so it is the frequency that carries the information.

"What is a bounded context?"

> The boundary within which one model and its language are consistent. The example that makes it concrete is the word customer: to sales it is a prospect with a pipeline stage, to billing it is a payment method and an address, and to support it is a history of tickets. Building one customer model for all three produces a class with forty fields where each consumer uses eight, and which cannot change without everyone agreeing. Three separate models, one per context, each small and each correct where it lives, referring to the same person by identifier and sharing nothing else, is the answer. It matters architecturally because bounded contexts are the best candidates for service boundaries, so it connects directly to where a system gets split.

"Explain CQRS, and when it is worth it."

> Command query responsibility segregation means using a different model for writing than for reading. The minimal version is two models and two code paths in the same database, and the full version has a separate read store updated asynchronously and shaped for the queries it serves. It pays when the read and write shapes genuinely diverge, because a normalised write model that protects invariants is often a poor shape for a dashboard needing six joins, and denormalising the write model to suit the dashboard damages what the write model is for. The costs are two models to maintain and, if the read store updates asynchronously, eventual consistency that users can see, so someone saves a thing and does not find it in the list, which needs a product answer rather than a technical one. Two things worth clarifying because people assume them: CQRS does not require event sourcing, and it does not require separate databases.

"When would you use event sourcing?"

> When the sequence of events is the truth rather than a derivative of it, so accounting or trading, and particularly where reconstructing historical state is a core requirement. I would be careful about the audit argument, because a regulatory audit obligation on its own does not imply event sourcing, since an append-only audit table or database change history usually satisfies it far more cheaply. What it gives is a complete audit log by construction rather than written alongside, the ability to reconstruct state at any past moment, and the ability to build a new projection retroactively for a question nobody thought to ask. The costs are larger than most teams expect: there is no table to query so every read shape needs a projection, event schemas have to stay readable forever so versioning becomes permanent work, snapshots are needed once aggregates have thousands of events, and deleting data conflicts with an append-only log, which collides with data protection rules requiring erasure. Outside those domains I would ask what problem it is solving, because an audit table and a change log usually deliver the actual requirement for a fraction of the cost.

"How do you stop an architecture eroding?"

> By putting the rules in the build rather than in a document, because a rule in a document decays and one in the build does not. Erosion happens through ordinary pressure, one reasonable shortcut at a time, and it is invisible until someone tries to make the change the architecture was supposed to make easy. Fitness functions are the mechanism: automated checks that the architecture still holds, run like any other test. In Java that is ArchUnit, so a rule that the domain package must not depend on infrastructure becomes a failing build rather than a code review comment, and the same applies to modules not reaching into each other and to packages being free of cycles. They are not only structural either, so a latency budget or a limit on deployable size works the same way. The underlying idea is evolutionary architecture: make reversible decisions quickly, make irreversible ones carefully, and spend effort turning the second kind into the first.

"How would you migrate a legacy system?"

> With a strangler fig rather than a rewrite. Put a facade in front of the existing system so callers stop depending on it directly, pick one capability with a clean seam, build it in the new system, route a fraction of traffic and verify the results match, then route all of it and delete the old path, and repeat. The reason this beats replacing everything at once is that a full rewrite produces no value until it finishes, competes with a system that keeps changing underneath it, and puts all the risk at the end, which is the worst place for it. The part I would plan hardest is data, because during the transition both systems usually need the same data, and the options are shared tables which couples them, synchronisation which risks divergence, or making one side the owner and the other a consumer of its events, which is cleanest and slowest.

20. Common mistakes

- Treating architecture as a phase that finishes before implementation starts, rather than a set of decisions revisited as information arrives.
- Applying hexagonal architecture to a service that is mostly create, read, update and delete, where the mapping layers cost more than they return.
- Splitting into microservices for technical reasons when the problem is organisational, and getting the distributed systems cost with none of the deployment independence.
- Splitting before deployment automation, tracing and on-call exist, which produces a distributed monolith.
- Drawing service boundaries along technical layers, so every request passes through several services and no feature fits in one.
- Building one shared model across contexts, which produces a class nobody can change and everybody depends on.
- Making aggregates large, so the transaction boundary locks far more than the change needed.
- Letting a third-party or legacy model into the domain instead of translating at the boundary with an anticorruption layer.
- Adopting event sourcing for the audit trail alone, when an audit table would have met the requirement at a fraction of the permanent cost.
- Assuming CQRS requires event sourcing or separate databases, and taking on both when only separated models were needed.
- Writing architecture rules in a document and expecting them to hold, rather than enforcing them in the build.
- Writing an ADR that lists only benefits, which tells a future reader nothing about when the decision should be reconsidered.
- Drawing one diagram for every audience, so it is too detailed for stakeholders and too vague for engineers.
- Designing an architecture that cuts across team boundaries, which Conway's Law will erode back toward the organisation chart.

21. Quick reference

Term	Meaning
Architecture	The decisions that are expensive to change later
Quality attribute	A measurable non-functional property, traded against others
ADR	A numbered record of one decision, its context and its consequences
Layered	Dependencies generally point downward. Traditional forms couple business logic to persistence
Hexagonal	Dependencies point inward. The application core owns the ports, infrastructure provides the adapters
Modular monolith	One deployable, strict module boundaries, no shared tables
Distributed monolith	Services that must be deployed together. The worst of both
Bounded context	The boundary within which one model and its language are consistent
Ubiquitous language	One vocabulary shared by engineers and domain experts, used in code
Aggregate	A cluster of objects treated as one consistency boundary, with a single root
Entity against value object	Identity that persists, against defined entirely by attributes
Anticorruption layer	Translation at a boundary, so another model does not enter the domain
Event notification	The event says something happened; consumers call back for detail
Event-carried state transfer	The event carries the data, so consumers need not call back
CQRS	Separate models for reading and writing. Not necessarily separate stores
Event sourcing	Events are the stored state; current state is derived by replay
Conway's Law	Systems copy the communication structure of the organisation building them
Inverse Conway manoeuvre	Change team structure to get the architecture wanted
Cognitive load	The real limit on how much one team can own
Fitness function	An automated check that the architecture still holds
Architecture erosion	The growing gap between the intended architecture and the real one
Strangler fig	Migrate capability by capability behind a facade, then delete the old system
C4	Context, container, component, code. Four levels of diagram zoom

Question	Where to start
Should this be one service or several?	Team structure first, then bounded contexts
Where does this boundary go?	Where the language changes, and where data ownership falls
Should these two things be consistent immediately?	If yes, consider one aggregate. If no, separate aggregates, and the service boundary is a separate decision
Why was this decided?	The ADR, and if there is not one, that is the finding
Is the architecture still intact?	A fitness function in the build, not a review comment
How do we replace this legacy system?	A facade, then one capability at a time, with data planned first
Which diagram do we need?	Context for stakeholders, container for engineers

Where the detail lives	Note
SOLID, coupling, cohesion, composition	Design principles
Adapter, Strategy, Observer and the rest	Design patterns
Requirements, estimation, building blocks, worked designs	System design
Sagas, outbox, consistency models, tracing	Distributed systems
REST, gRPC, versioning, layering a Spring service	API and backend development